

RoundGuard: Exact Attainable Discrepancies in Monetary Rounding Policies

Kun Huang

Economics and Management School, Wuhan University

huangkun123huang@163.com

Abstract

Exact decimal arithmetic does not determine when a monetary calculation should be rounded. A line-item rule and its aggregate counterpart can produce different payable amounts even when both use exact rational intermediates. Universal rounding bounds do not tell a developer whether their endpoints are attainable on the permitted amount grids. We present RoundGuard, an exact analyzer for explicitly declared monetary rounding policies. For rational affine expressions with translation-compatible quantizers, it computes structural shifts and compares policies on feasible residue representatives. For independent affine line items, a cyclic residue dynamic program (DP) computes attainable signed discrepancy extrema and reconstructs inputs achieving them, without enumerating the Cartesian input grid. Its operation count is pseudo-polynomial in a common denominator; sign-sensitive modes require a proved nonnegative-line condition or an unreduced satisfiability-modulo-theories (SMT) fallback. A JavaScript Object Notation (JSON) rule interface supports six rounding modes, finite placement families, and bounded piecewise expressions. Independent exhaustive decimal evaluation checks the analyzer on synthetic, canonical, and public-rule-derived fragments. Scaling experiments compare the dynamic program with full-grid SMT and a residue-reduced SMT ablation. The contribution is a specialized analysis of feasible monetary discrepancies built on established shift-periodic arithmetic and min/max-plus dynamic programming, rather than a new general theory of rounding.

1 Introduction

Rounding stages are observable parts of a monetary rule. The UK HM Revenue & Customs (HMRC) invoice guidance distinguishes line calculations from rounding the final value-added tax (VAT) total [14]. The US Internal Revenue Service (IRS) describes adjustments arising when payroll taxes are rounded during individual payroll computations [15]. US Social Security Administration (SSA) guidance prescribes different cent, dime, and dollar stages, and explicitly requires certain entitlement comparisons before lower-dollar rounding [22]. These examples concern specified discrete outcomes, not merely the numerical approximation of a real-valued formula.

The arithmetic foundation is already substantial. Fluctuat and Daisy analyze finite-precision computations, including fixed-point arithmetic [8, 11]; FPTaylor derives rigorous roundoff bounds [23]. Catala gives executable legal specifications [16]. Its date analysis proves insensitivity to alternative calendar rounding conventions [18], while CUTEcat encodes rational monetary rounding in Z3 and reports backend discrepancies [13]. Therefore neither monetary rounding semantics nor counterexample generation alone constitutes the research gap addressed here.

Our narrower question is: *what discrepancy can actually occur between declared monetary rounding policies on specified finite amount grids?* A universal bound treats operands as arbitrary rationals. A program's rates and permitted tick values can make extremal residuals impossible. Exhaustive evaluation is definitive but its input product grows with both the amount ranges

and the number of lines. Generic exact integer reasoning can express the problem, but does not automatically exploit independent line structure.

We develop a specialized exact analysis with four components. First, structural shift certificates give a feasible representative domain for common-drift rounding policies. This instantiates established shift-periodic Chvátal and quasi-affine arithmetic [1, 24, 25]; periodicity itself is not claimed as new. Second, an attainable-extrema theorem identifies sufficient cyclic states for line-versus-total comparison and provides witness reconstruction, including a nonnegative extension for sign-sensitive modes. Third, a runnable analyzer dispatches between this dynamic program, residue-reduced SMT, and unreduced bounded SMT. Fourth, an openly specified benchmark records independent oracles, executable diagnostics, and method ablations. The results characterize this restricted analyzer; they do not certify a deployed tax or payroll system.

2 Examples and the analysis contract

Amounts in the examples are measured in cents. Let $Q_{q,m}$ round a rational to the zero-anchored grid $q\mathbb{Z}$ using mode m . With two raw line taxes $y_1 = y_2 = 1/2$, nearest-even line rounding produces zero, whereas nearest-even total rounding produces one cent:

$$P_{\text{line}} = \text{he}(1/2) + \text{he}(1/2) = 0, \quad P_{\text{total}} = \text{he}(1) = 1.$$

The distinction persists without binary floating-point arithmetic. Conversely, reassociating addition of values already on one exact monetary grid does not create rounding errors: every sum remains on that grid. RoundGuard includes this insensitive case rather than treating any rounded addition as a warning.

An input grid matters. The sharp nearest-even aggregation bound for an even number n of arbitrary rational operands is $nq/2$. However, a line $y = x/5$ with integer x cannot have a half-integer normalized value. Several such lines can prevent the universal extreme. A useful diagnostic must distinguish an unattainable envelope from a discrepancy reached by a valid input. The batch analysis retains feasible residues and returns an attaining assignment.

The user supplies a nonempty finite box of integer ticks, exact input quanta, and a finite family $\mathcal{P}$ of expression variants. The observation is the final rational amount, or a rational-valued branch result. We ask whether

$$\forall x \in X \ \forall P, Q \in \mathcal{P}, \quad P(x) = Q(x), \quad \Delta(\mathcal{P}, X) = \max_{x \in X} (\max_{P \in \mathcal{P}} P(x) - \min_{P \in \mathcal{P}} P(x)).$$

Completeness is relative to the supplied box and declared policy family. Optional sites enumerate round/no-round placements explicitly; RoundGuard does not infer whether a placement is legally admissible. A sensitive result does not select the correct policy or justify an automatic repair.

3 Language and exact semantics

The certified core consists of finite expressions

$$e ::= c \mid x_i \mid e + e \mid e - e \mid re \mid Q_{q,m}(e), \\ c, r \in \mathbb{Q}, \quad q \in \mathbb{Q}_{>0}, \quad m \in \{\text{FLOOR}, \text{CEILING}, \text{HALF_EVEN}\}.$$

An input is $s_i x_i$, where $x_i \in \mathbb{Z}$ and the declared positive rational s_i is its quantum. Coefficients below are measured with respect to integer ticks, so they already include s_i . The interface labels grids as Money, Integer, Decimal, or Rate; Integer has unit quantum. These labels document input representation, rather than implement a currency or dimensional type system. Rates used in multiplication are exact constants. Products of two inputs, input-dependent divisors or quanta, overflow, and loops are unsupported.

For a normalized argument $t = y/q$, FLOOR and CEILING choose $\lfloor t \rfloor$ and $\lceil t \rceil$. HALF_EVEN chooses a nearest integer, resolving a tie toward the even candidate. HALF_UP follows the conventional decimal convention of nearest with ties away from zero. DOWN truncates toward zero; UP rounds away from zero. Multiplying the selected integer by q gives the result. Rational constants are integers or strings such as "0.062" and "1/3"; JSON floating-point quanta and Python floating literals are rejected.

The implementation also supports `min`, `max`, and numeric conditionals with ordered comparisons. These are encoded exactly in bounded SMT but lie outside the general shift certificate. The following fragment generates the line and total variants automatically:

```
{
  "variables": {
    "x": {"min": 0, "max": 19},
    "y": {"min": 0, "max": 19}
  },
  "batch": {
    "lines": ["x/5", "y/5"],
    "quantum": "1",
    "mode": "HALF_EVEN"
  }
}
```

Expressions are parsed through Python's syntax tree, not executed as Python. Attribute access, arbitrary function calls, and nonlinear arithmetic are rejected. A JSON template can contain `maybe(e,q,m,site)`; shared site names make a correlated on/off decision, and distinct names enumerate independent choices. At most ten sites are enumerated. This is a finite policy interface rather than a source analyzer for arbitrary Python or Catala programs.

4 Structural shifts and exact box reduction

Erase each quantizer to obtain $\bar{e}(x) = b_e + \sum_i a_{e,i}x_i$. FLOOR and CEILING commute with integer grid shifts; nearest-even commutes with even grid shifts:

$$\mathcal{Q}_{q,m}(y + p_m k q) = \mathcal{Q}_{q,m}(y) + p_m k q, \quad p_m = \begin{cases} 1 & m \in \{\text{FLOOR}, \text{CEILING}\}, \\ 2 & m = \text{HALF_EVEN}. \end{cases} \quad (1)$$

Both statements hold for negative k . An odd shift does not suffice for nearest-even: $\text{he}(1/2) = 0$ and $\text{he}(3/2) = 2$.

For every coordinate, inspect all quantizer nodes $v = \mathcal{Q}_{q_v, m_v}(u_v)$ in the family and define

$$T_i = \text{lcm}_v \text{den} \left(\frac{a_{u_v, i}}{p_{m_v} q_v} \right). \quad (2)$$

The denominator is reduced, and the empty lcm is one. These shifts are sufficient, not necessarily minimal.

Proposition 1 (Common-drift reduction). *For a family in the certified core, every expression satisfies $e(x + T_i \mathbf{e}_i) = e(x) + a_{e,i} T_i$. If the variants have equal erased slope vectors, their pairwise differences are periodic with these coordinate periods. On $X = \prod_i \{L_i, \dots, U_i\}$, all discrepancy values occur on*

$$H = \prod_i \{L_i, \dots, L_i + \min(T_i, U_i - L_i + 1) - 1\}.$$

Every witness found on H is feasible on X .

Proof. Constants and variables have their stated affine drift. Addition, subtraction, and constant scaling preserve it. At a quantizer, induction gives child drift $a_{u,i} T_i$. Equation (2) makes that drift a permitted multiple in (1); quantization preserves it. Subtracting two equal-slope drift identities yields periodic discrepancy. Map any coordinate to $L_i + ((x_i - L_i) \bmod T_i)$. If the original width is smaller than T_i , this is the original coordinate; otherwise it lies in the first period. The mapped input lies in H , differs by period multiples, and preserves each discrepancy. Since $H \subseteq X$, the attainable value sets coincide. $\square$

This proposition operationalizes known shift-periodic arithmetic, with explicit mixed-mode parity and feasible box representatives. It is not an efficiency guarantee for a general graph: $|H|$ can be a large product, and coefficient denominators can make T_i large. Unequal slopes trigger unreduced SMT. HALF_UP, DOWN, and UP also trigger that fallback: they do not have global signed translation covariance. For example, HALF_UP maps $-1/2$ to -1 and $3/2$ to 2 , violating even-shift covariance across zero. Nonnegative inputs alone would be insufficient if a negative intermediate expression were permitted.

5 Attainable extrema for independent lines

The specialized batch path avoids the Cartesian product of line residues. Normalize a common quantum q to one and let

$$y_i = a_i x_i + c_i, \quad x_i \in \{L_i, \dots, U_i\}, \quad P_{\text{total}} = R_0 \left(\sum_i y_i \right), \quad P_{\text{line}} = \sum_i R_i(y_i).$$

Each line has its own independent tick input. Constant lines are allowed. In the theorem, R_i and R_0 may be FLOOR, CEILING, or HALF_EVEN. The implementation uses a fixed common mode for a batch. It rejects repeated line-input dependencies rather than treating them as independent.

Choose an integer $B > 0$ clearing the denominators of all normalized a_i, c_i . Put $A_i = Ba_i$, $C_i = Bc_i$, $z_i = A_i x_i + C_i$, $M = 2B$, and

$$d_i = BR_i(y_i) - z_i, \quad h(r) = BR_0(r/B) - r \quad (0 \leq r < M).$$

The local pair $(z_i \bmod M, d_i)$ is invariant under the sufficient shift

$$T_i = \frac{M}{\gcd(|A_i|, M)} \leq M, \quad \gcd(0, M) = M.$$

Indeed, that shift changes z_i by a multiple of $2B$ and y_i by an even integer. Enumerating the first $\min(T_i, U_i - L_i + 1)$ feasible ticks therefore obtains every local pair. For each attainable residue t , store the minimum $\ell_i(t)$ and maximum $u_i(t)$ of d_i , together with local witnesses. Absent residues remain unreachable. The interval endpoints do not imply that every intervening error is attainable.

Initialize only $V_0^-(0) = V_0^+(0) = 0$. With cyclic residue indices, update

$$V_k^-(r) = \min_{s+t \equiv r \pmod{M}} (V_{k-1}^-(s) + \ell_k(t)), \quad (3)$$

$$V_k^+(r) = \max_{s+t \equiv r \pmod{M}} (V_{k-1}^+(s) + u_k(t)), \quad (4)$$

using reachable states only. This is ordinary cyclic min/max-plus convolution; the application-specific point is that these two endpoints per residue suffice for attainable monetary discrepancy extrema.

Theorem 2 (Exact batch discrepancy). *Under the independent affine-line premises, the exact signed extrema of $P_{\text{total}} - P_{\text{line}}$ are*

$$\delta_{\min} = \min_{r \text{ reachable}} \frac{h(r) - V_n^+(r)}{B}, \quad \delta_{\max} = \max_{r \text{ reachable}} \frac{h(r) - V_n^-(r)}{B}.$$

Both endpoints are attained by reconstructible in-box inputs. Equivalence holds exactly when both are zero. The largest absolute discrepancy is $\max(-\delta_{\min}, \delta_{\max})$. Original monetary units multiply the extrema by q .

Proof. Induct on the prefix length. Every feasible prefix assignment decomposes into a feasible previous prefix and a feasible local choice. Conversely every such pair combines because the input domains are independent. At fixed residues, the least or greatest additive error is the sum of the corresponding attained endpoints. Minimizing or maximizing over residue decompositions proves (3)–(4), including attainment.

For a full assignment write $S = \sum_i z_i$, $D = \sum_i d_i$, and $r = S \bmod M$. Then

$$B(P_{\text{total}} - P_{\text{line}}) = BR_0(S/B) - S - D = h(r) - D.$$

The last identity follows because $S/B - r/B$ is an even integer. At a fixed residue the objective decreases with D , so the two attained error endpoints suffice. Taking extrema over reachable residues proves the formulas. Store the predecessor residue and chosen local tick at every retained endpoint. Following those pointers recovers an assignment made entirely of feasible local witnesses, attaining the selected final endpoint. $\square$

Local construction costs $O(nM)$ arithmetic operations, and dense convolution costs $O(nM^2)$. Two prefix value arrays require $O(M)$ working storage. Per-layer backpointers require $O(nM)$ records; witness reconstruction costs $O(n)$. Sparse implementation work is proportional to the reachable-prefix and local-residue products. These are arithmetic-operation bounds, excluding coefficient preprocessing and operand bit costs. They are pseudo-polynomial in B , which is an lcm and can grow exponentially in the binary input description. The prototype declines the DP path when $M > 400$.

Proposition 3 (Nonnegative sign-sensitive extension). *Theorem 2 also holds for HALF_UP, DOWN, and UP when every raw normalized affine line is nonnegative throughout its declared interval.*

Proof. On nonnegative arguments these modes equal $\lfloor t + 1/2 \rfloor$, $\lfloor t \rfloor$, and $\lceil t \rceil$, respectively. Integer translation preserves their errors whenever both arguments remain nonnegative. All local representatives lie inside their original domains. Each raw line, the actual total S/B , and the normalized representative total r/B are nonnegative. Thus every local-pair and outer-error identity used in Theorem 2 remains valid; the independent convolution proof is unchanged. $\square$

The implementation checks both affine endpoint values of every raw line, not only the signs of the inputs. Otherwise it falls back. Neither this proposition nor the general box reduction discards cross-line feasibility constraints. A fixed grand total or a shared discount input requires another encoding; the current batch interface does not support coupled constraints.

6 SMT fallback and diagnostic guarantees

We encode integer input ticks, rational arithmetic, floor casts, tie comparisons, and integer parity in Z3 [10]. An expression has a statically fixed rational output denominator: addition takes an lcm, constant scaling multiplies denominator bounds, and a quantizer returns an integer times its quantum. Therefore multiplying a discrepancy by a common denominator gives an integer objective.

The analyzer first asks whether any pair differs. UNSAT yields SAFE within the declared scope. SAT yields a concretely replayed witness. For exact extrema, it binary-searches the nonnegative integral family spread between an attained lower bound and a sound upper envelope. For common full erasures, propagated absolute quantization errors give an envelope; otherwise ordinary rational range intervals do. Each successful threshold query supplies a model. Completed search supplies an optimum with a replayed attaining input. A solver timeout produces UNKNOWN for detection, or SENSITIVE with unestablished maximum and valid lower/upper bounds after a witness has already been found. Neither is reported as an equivalence proof or an exact optimum.

Proposition 4 (Conditional analyzer correctness). *Assuming the implemented concrete and symbolic operators agree with the stated semantics, and successful solver answers are correct, every completed SAFE result implies equality of the declared variants over the input box. Every completed maximum is the attainable family spread on that box. The DP path requires the premises of Theorem 2 or Proposition 3; the reduced SMT path requires Proposition 1.*

Proof. Residue reduction preserves discrepancy value sets, while the batch DP preserves their attained extrema and the zero-discrepancy test. An unreduced SMT query is the direct counterexample formula for the finite box. After a SAT query, a sound upper bound and integer objective make threshold feasibility monotone; binary search terminates at the largest feasible objective. Replaying its model establishes attainment. UNSAT on the difference formula establishes equality. Unknown queries do not justify either conclusion. $\square$

This is an algorithmic argument under implementation and solver assumptions, not a mechanized verification of Python or Z3. The proof ledger labels mathematical results PROVED and executable enumeration as bounded sanity checks. No result is labelled MACHINE-CHECKED by a proof assistant.

7 Evaluation

Questions, corpus, and protocol. We ask whether completed diagnostics agree with independently executed small-grid semantics, whether residue reduction and the specialized DP change exact-search behavior, and whether a universal envelope is attained on the declared grids. The deterministic generator uses seed 20261003. The effectiveness corpus contains 48 synthetic programs, 12 canonical examples, and six public-rule-derived fragments: 66 programs in total, of which the oracle classifies 50 as sensitive and 16 as safe. The corresponding group counts are 37/48, 8/12, and 5/6 sensitive. Each program has 1–3 tick variables, two to four policies, and 21–1600 input assignments. Counting repeated syntax separately across policies gives 6–28 abstract syntax tree (AST) nodes (median 13); maximum root-to-leaf depth is 3–5 (median 4), with the root counted as depth one. These are small expression benchmarks rather than production programs.

The exhaustive oracle evaluates all 19,940 assignments in this corpus using Python Decimal’s six rounding conventions, separately from the analyzer’s Fraction quantizer and SMT encoding. Terminating rationals are converted by exact digit-tuple construction, independent of Decimal’s current precision. The precision budget includes denominator powers of two and five and intermediate decimal spans. Inexact and rounded arithmetic traps reject silent approximation; only the explicitly requested grid quantization may round. Nonterminating conversions or normalized quantum divisions are rejected. All saved corpus computations complete under these guards. The oracle shares the parsed AST with the analyzer, so this agreement tests arithmetic and analysis after parsing; it does not independently establish every parser translation. Saved `ground_truth.json` records each oracle maximum and witness, and `diagnostics.json` records method outputs.

RoundGuard’s automatic dispatch is compared with the same symbolic encoder on the full box, 32 uniformly sampled assignments, Hypothesis search configured with `max_examples=64`, and independent range intervals for each policy. Sampling stops at the first differing assignment; Hypothesis may also shrink a found example, so 64 is a configuration parameter rather than a matched wall-time or strict total-evaluation budget. A testing result with no witness does not prove equality. Interval warnings are counted against the exact label, but the interval baseline is a deliberately simple correlation-losing envelope, not a state-of-the-art numerical analyzer. The effectiveness measurements have one execution per program and method; their medians summarize this heterogeneous corpus rather than repeated-run timing uncertainty.

Method	Cases	FP	Missed	Unknown	Median ms
RoundGuard	66	0	0	0	9.77
Full-grid SMT	66	0	0	0	14.30
Random (32)	66	0	2	0	2.83
Hypothesis (64)	66	0	1	0	119.52
Independent intervals	66	15	0	0	0.28

Table 1: Effectiveness on 66 programs: 50 sensitive and 16 safe. FP (false positive) counts warnings on oracle-safe programs; Missed counts oracle-sensitive programs without a reported witness or warning. A testing miss is not a false proof of safety. Exact-method maxima also agree with the exhaustive oracle. Median wall times are across programs.

Small-grid agreement and detection. Both exact methods complete on all 66 programs, match every oracle maximum, and have no false warnings or missed sensitive cases (Table 1). Automatic dispatch selects residue DP for 22 programs, reduced SMT for 19, and unreduced SMT for 25. Its median wall time is 9.77 ms, compared with 14.30 ms for full-grid SMT. Random testing misses two threshold programs, and Hypothesis misses one currency-conversion program. Independent intervals warn on 15 of the 16 safe programs. These results establish agreement on the saved cases; neither the mixture nor the small miss counts estimates defect prevalence in financial software.

Scaling and method ablations. The scaling suite has 24 independent-line cases: $n \in \{2, 4, 8, 16, 32, 64\}$, tick bounds $0 \leq x_i \leq U$ with $U \in \{3, 20, 10^4, 10^9\}$, and cyclic rates $1/2, 1/4, 1/5, 1/10$. All use cent HALF_EVEN rounding. The modulus is $M = 8$ for two lines and $M = 40$ otherwise; this suite varies line count and amount-window width, not large coefficient denominators. The largest box contains $(10^9 + 1)^{64}$ assignments. We execute each case and each of DP, reduced SMT, and full-grid SMT three times, giving 216 saved runs. Reduced versus full SMT isolates structural box reduction within one encoder; reduced SMT versus DP evaluates the additional independent-line recurrence.

All methods are timed from analysis entry through the returned diagnostic with `perf_counter`; JSON loading and oracle construction are excluded, while method-specific preprocessing, encoding, optimization, and replay are included. The solver limit is 1500 ms *per check*, including the initial detection check and each binary-search query. Consequently a complete or partially optimized run can take longer than 1.5 s. This is not an equal total-run budget, and the DP has no matching timeout. Measurements come from one Windows machine with Python 3.11.5 and Z3 4.15.3; package versions and protocol settings are saved in `environment.json`.

DP establishes the exact maximum in 72/72 runs, reduced SMT in 35/72, and full-grid SMT in 19/72. Among unfinished reduced-SMT runs, 25 return UNKNOWN before finding a witness and 12 report sensitivity with an unestablished maximum. The corresponding full-grid counts are 36 and 17. Both outcomes are distinct from either a proof of equality or an exact optimum, and their separate statuses remain in the raw records.

The largest DP median over the 24 cases is 101.94 ms (Figure 1). For the four-line, $U = 10^9$ cell, the SMT medians are 81.83 ms with structural reduction and 1953.54 ms on the full grid. Reduction is not guaranteed to improve solver wall time: encoding and solver overhead can change its practical effect. Cells containing unfinished runs do not support exact-answer speedup ratios. The DP’s finite residue work explains its weak dependence on interval width here, while larger denominators, coupled inputs, or branch structure require separate evaluation. Runtime and completion numbers are generated from the saved records through `result_macros.tex`; reruns on another machine require reexamining the resulting plot and outcomes.

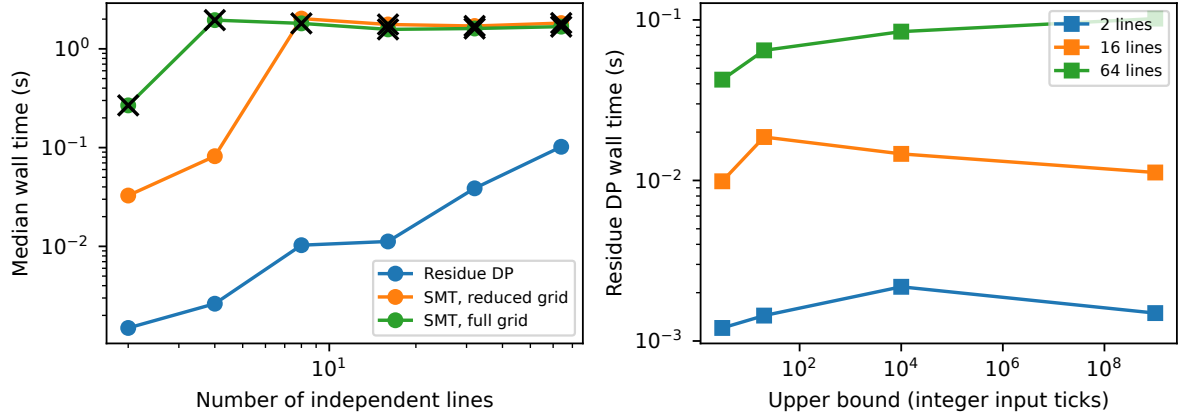


Figure 1: Saved scaling measurements. Left: median observed wall time over three runs at $U = 10^9$; a cross marks a cell with at least one unfinished optimum computation. Such points measure time to the returned diagnostic and must not be interpreted as time to an exact answer. Right: DP timing as amount-window width changes for selected line counts. Coefficient denominators remain fixed within these curves.

Independent scaling certificates and bound quality. Large boxes are not exhaustively enumerated. For each line, an independently executed Decimal table covers its feasible local period, or its whole interval when shorter; the sufficient periods of these benchmark rates are at most 20 ticks. Summing the local error minima and maxima gives a sound envelope for the total line error. The outer nearest-rounding error is at most $1/2$ cent. Since the final discrepancy is an integer number of cents, flooring this envelope supplies an independent integral upper bound. A DP-produced input is checked against every declared tick bound and replayed through Decimal. Its attained spread equals that independently derived upper bound in all 24 cases, certifying the maximum without assuming the DP’s optimum calculation. The certificate values and witnesses are saved in `scaling_ground_truth.json`. This construction is specific to the saved scaling family, not an independent solver for arbitrary input programs.

The unrestricted nearest-even envelope is $\text{he}(n/2)$ cents. At 64 lines and $U = 3$, RoundGuard finds an attainable maximum of 27 cents, compared with the universal 32-cent envelope. At $U \geq 20$, the 64-line maximum is 30 cents, leaving a two-cent gap even with complete local-period coverage; the heterogeneous line grids still exclude the universal endpoint. For 32 lines the corresponding maxima are 14 and 15 cents, compared with 16 cents universally. The saved upper-envelope certificate is tight in all scaling cases, whereas the universal envelope is strictly larger in ten of the 24 cases. This difference measures attainable-bound precision on declared grids, rather than a change in rounding semantics.

8 Public-rule-derived fragments

The benchmark includes independently written fragments motivated by publicly documented rounding stages. They are deliberately small and contain explicit counterfactual policies; they are not complete legal calculators. Table 2 gives the attained spread on the grids saved in each JSON file.

The HMRC fragment compares line VAT rounded down to tenths of a penny, followed by final penny rounding, with final-only penny rounding [14]. The two-line 20% rate and narrow amount windows are benchmark assumptions, not an assertion that every policy applies to every trader. Payroll uses the separate 6.2% and 1.45% employee-side rates appearing in the IRS fractions-of-cents discussion and compares them with a deliberately combined-rate calculation [15].

Fragment	Maximum (cents)	Method
cash_settlement	5	residue-dp
hmrc_invoice	0	residue-smt
irs_payroll	1	smt
rational_money	1	smt
ssa_benefit	66	residue-smt
ssa_entitlement	50	smt

Table 2: Exact discrepancy on six rule-derived fragments. Values are cents. A difference establishes sensitivity of the stated comparison, not a defect in the cited rule or deployed software.

It ignores wage caps, Additional Medicare Tax, and reporting adjustments. Shared wages make this a coupled expression, handled by SMT rather than independent-line DP.

The benefit fragments isolate the SSA’s prescribed dime/dollar stages and the order of entitlement comparison [22]; the chosen adjustment factor, offset, and entitlement threshold are illustrative inputs, not a full statutory benefit formula. Cash settlement compares final nickel rounding with an intentionally early per-component alternative motivated by Canadian guidance to round the final bill after taxes [5]. The software fragment compares an intermediate-rounded rational calculation with deferred rounding, motivated by Brick Money’s documented RationalMoney workflow [3]. Its rates are independently chosen, and no external implementation is copied.

9 Related work

Skala’s non-peer-reviewed preliminary draft studies consistent rounding of financial totals [21]. It gives algorithms to choose rounded labels for a fixed totals forest, including minimax node-error optimization. Its line-versus-total motivation is directly adjacent to ours. RoundGuard instead fixes the policies and optimizes their discrepancy over permitted input grids; the different objective makes Skala’s algorithms unsuitable as same-task runtime baselines.

Shift-periodic nested rational ceiling expressions already appear in Williams’ account of Chvátal functions [25]. Basu et al. formalize affine Chvátal trees and mixed-integer representability [1]. The isl library manipulates exact integer sets and quasi-affine expressions [24]; Presburger functions and counting have broader structural treatments [26]. RoundGuard specializes these foundations to feasible representatives for prescribed monetary policies. It does not offer a new decidability result or a general competitor to integer-set libraries.

The cyclic extrema recurrence belongs to the established min/max-plus dynamic-programming pattern. Work on near-convex min-plus convolution accelerates structured knapsack instances [4]; our residue tables need not satisfy those additional premises, and the prototype uses direct sparse convolution. Low-discrepancy sequence rounding also uses compact state graphs and optimization [20]. It chooses rounded outputs for a fixed sequence, whereas our batch formulation fixes rounding policies and optimizes over feasible amount grids. This distinction limits the claimed application-specific contribution.

Numerical analyzers such as Fluctuat, Daisy, FPTaylor, and Gappa provide sound error analysis or certificates [8, 9, 11, 23]. Fixed-point synthesis studies implementation accuracy [7]; Flocq supplies foundational arithmetic proofs [2]. Herbie searches expression rewrites using sampled accuracy estimates [19], and robustness analyses handle discontinuous branch changes [12]. These systems address richer numerical programs than our exact affine batch fragment. RoundGuard’s different contract is an attained discrepancy between two prescribed discrete policies, rather than an approximation guarantee against an ideal real formula.

Catala’s semantics and verified compilation address translating computational law [16]; the date work already treats rounding-insensitivity as a relational property [18]. CUTECat’s rational monetary encoding and backend discrepancy findings are direct prior art for our fallback and motivation [13]. Recent recommendations for public administrations likewise emphasize exact monetary representation and testing [17]. The present work adds a restricted attainable-extrema path, and provides no Catala integration. FPBench motivates auditable benchmarks for numerical tools [6]; our JSON monetary suite is separate and is not claimed as an extension of FPBench.

10 Limitations and validity

The main risk to practical generalization is independence. Many payroll and benefit computations share input amounts, use piecewise bands, or constrain an invoice’s grand total. Those programs use the bounded fallback, or require constructs outside this implementation. The prototype supports only one active tick variable per affine DP line and one common rounding mode, despite the theorem allowing different covariant local modes. Generic optional placements are explicitly enumerated rather than compressed by the batch theorem. Large denominators can eliminate the DP advantage; full-period enumeration is not polynomial in coefficient bit size.

The evaluation is a controlled microbenchmark study on one Windows machine. Its generated amounts are not sampled transaction data, and the six public-rule-derived cases are simplified fragments. The suite demonstrates consistency against independent exact oracles, not prevalence of defects in financial software. Both SMT paths share the same encoder, so their comparison isolates domain reduction and batch structure rather than ranking independently developed solvers. Timeouts are per check, and do not create one equal total-run budget. Runtime depends on solver version, scheduling, and machine load; seeds fix inputs and search configuration but cannot fix wall time.

The mathematical proofs are ordinary proofs independently reviewed for their premises. Enumeration and property tests provide additional bounded checks; they cannot substitute for a proof. Decimal oracle comparisons share the parsed tree and therefore do not independently validate the parser’s entire translation. Parser rejection and known-output integration tests mitigate part of that risk. Source rules were read on the stated research date; rule changes require revisiting the fragments. No diagnostic decides legal compliance, chooses a preferred policy, or verifies the author’s institutional software.

11 Conclusion

RoundGuard computes feasible rounding-policy discrepancies in a deliberately restricted exact monetary model. Structural periods reduce amount grids when drift cancels, while a cyclic extrema dynamic program avoids multiplying independent line domains. The analysis returns attained extrema and concrete witnesses, with explicit parity, sign, independence, and denominator conditions. The implementation and benchmark make those conditions observable and route unsupported cases to bounded SMT. Extending the useful batch path to coupled domains and validating it on larger source programs remain open work.

Availability and assistance. Code, inputs, results, proofs, and reproduction scripts are available at <https://github.com/beibeihk/roundguard>. AI tools assisted literature triage, implementation, and manuscript drafting. Reported computations have executable artifact records; the proofs are not proof-assistant mechanizations.

References

- [1] Amitabh Basu, Kipp Martin, Christopher Thomas Ryan, and Guanyi Wang. Mixed-Integer Linear Representability, Disjunctions, and Chvátal Functions—Modeling Implications. *Mathematics of Operations Research*, 44(4):1264–1285, 2019. doi: 10.1287/moor.2018.0967. URL <https://arxiv.org/html/1711.07028v1>.
- [2] Sylvie Boldo and Guillaume Melquiond. Flocq: A Unified Library for Proving Floating-Point Algorithms in Coq. In *20th IEEE Symposium on Computer Arithmetic (ARITH 2011)*, pages 243–252, 2011. doi: 10.1109/ARITH.2011.40. URL <https://guillaume.melquiond.fr/doc/11-arith20-article.pdf>.
- [3] Brick contributors. brick/money: Money contexts and advanced calculations. <https://github.com/brick/money>, 2026. Accessed 3 October 2026. Mutable documentation accessed 3 October 2026; library semantics alone are not evidence for an implemented adapter or a discovered bug.
- [4] Karl Bringmann and Alejandro Cassis. Faster 0-1-Knapsack via Near-Convex Min-Plus-Convolution. In *31st Annual European Symposium on Algorithms – ESA 2023*, pages 24:1–24:16, 2023. doi: 10.4230/LIPIcs.ESA.2023.24. URL <https://arxiv.org/abs/2305.01593>.
- [5] Canada Revenue Agency. Phasing Out of the Penny: GST/HST Info Sheet GI-131. https://publications.gc.ca/collections/collection_2012/arc-cra/Rv12-5-131-2012-eng.pdf, 2012. Accessed 3 October 2026. Cash and electronic payment differ; this policy is not permission to round individual tax amounts to five cents.
- [6] Nasrine Damouche, Matthieu Martel, Pavel Panchekha, Chen Qiu, Alexander Sanchez-Stern, and Zachary Tatlock. Toward a Standard Benchmark Format and Suite for Floating-Point Analysis. In *Numerical Software Verification – NSV 2016, Revised Selected Papers*, volume 10152 of *Lecture Notes in Computer Science*, pages 63–77, 2017. doi: 10.1007/978-3-319-54292-8_6. URL <https://homes.cs.washington.edu/~ztatlock/pubs/fpbench-damouche-nsv16.pdf>.
- [7] Eva Darulova, Viktor Kuncak, Rupak Majumdar, and Indranil Saha. Synthesis of fixed-point programs. In *Proceedings of the 11th ACM International Conference on Embedded Software – EMSOFT 2013*, pages 22:1–22:10, 2013. doi: 10.1109/EMSOFT.2013.6658600. URL <https://lara.epfl.ch/~kuncak/papers/DarulovaETAL13SynthesisFixedPointPrograms.pdf>.
- [8] Eva Darulova, Anastasiia Izycheva, Fariha Nasir, Fabian Ritter, Heiko Becker, and Robert Bastian. Daisy - Framework for Analysis and Optimization of Numerical Programs (Tool Paper). In *Tools and Algorithms for the Construction and Analysis of Systems – TACAS 2018*, volume 10805 of *Lecture Notes in Computer Science*, pages 270–287, 2018. doi: 10.1007/978-3-319-89960-2_15. URL https://link.springer.com/chapter/10.1007/978-3-319-89960-2_15.
- [9] Marc Daumas and Guillaume Melquiond. Certification of bounds on expressions involving rounded operators. *ACM Transactions on Mathematical Software*, 37(1):2:1–2:20, 2010. doi: 10.1145/1644001.1644003. URL <https://guillaume.melquiond.fr/doc/09-toms.pdf>.
- [10] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems – TACAS 2008*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340, 2008. doi: 10.1007/978-3-540-78800-3_24. URL https://link.springer.com/content/pdf/10.1007%2F978-3-540-78800-3_24.pdf.

- [11] Eric Goubault and Sylvie Putot. Static Analysis of Finite Precision Computations. In *Verification, Model Checking, and Abstract Interpretation – VMCAI 2011*, volume 6538 of *Lecture Notes in Computer Science*, pages 232–247, 2011. doi: 10.1007/978-3-642-18275-4_17. URL <https://www.lix.polytechnique.fr/Labo/Sylvie.Putot/Publications/vmcai11.pdf>.
- [12] Eric Goubault and Sylvie Putot. Robustness Analysis of Finite Precision Implementations. In *Programming Languages and Systems – APLAS 2013*, volume 8301 of *Lecture Notes in Computer Science*, pages 50–57, 2013. doi: 10.1007/978-3-319-03542-0_4. URL <https://arxiv.org/pdf/1309.3910>.
- [13] Pierre Goutagny, Aymeric Fromherz, and Raphaël Monat. CUTEcat: Concolic Execution for Computational Law. In *Programming Languages and Systems – ESOP 2025*, volume 15695 of *Lecture Notes in Computer Science*, pages 31–61, 2025. doi: 10.1007/978-3-031-91121-7_2. URL https://rmonat.fr/data/pubs/2025/2025-05-03_esop_cutecat.pdf.
- [14] HM Revenue and Customs. VAT guide (VAT Notice 700). <https://www.gov.uk/guidance/vat-guide-notice-700>, 2026. Accessed 3 October 2026. Guidance conditions matter; examples do not prove both compared alternatives lawful for every trader. Last update observed 25 June 2026.
- [15] Internal Revenue Service. Publication 15 (2026), (Circular E), Employer’s Tax Guide. <https://www.irs.gov/publications/p15>, 2026. Accessed 3 October 2026. A restricted arithmetic model is not a complete implementation of payroll tax caps, additional taxes, adjustments or filing rules.
- [16] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: a programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP): 77:1–77:29, 2021. doi: 10.1145/3473582. URL <https://arxiv.org/abs/2103.03198>.
- [17] Raphaël Monat and Liane Huttner. Coding computational laws: 20 recommendations for public administrations. *International Review of Law, Computers & Technology*, pages 1–17, 2026. doi: 10.1080/13600834.2026.2628389. URL https://rmonat.fr/data/pubs/2026/2026-02-16_legal_code_recos.pdf. Published online 16 February 2026; publisher notes a subsequent correction.
- [18] Raphaël Monat, Aymeric Fromherz, and Denis Merigoux. Formalizing Date Arithmetic and Statically Detecting Ambiguities for the Law. In *Programming Languages and Systems – ESOP 2024*, volume 14577 of *Lecture Notes in Computer Science*, pages 421–450, 2024. doi: 10.1007/978-3-031-57267-8_16. URL <https://arxiv.org/html/2403.08935v1>.
- [19] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. Automatically Improving Accuracy for Floating Point Expressions. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation – PLDI 2015*, pages 1–11, 2015. doi: 10.1145/2737924.2737959. URL <https://homes.cs.washington.edu/~ztatlock/pubs/herbie-panchekha-pldi15.pdf>.
- [20] Kunihiro Sadakane, Nadia Takki-Chebihi, and Takeshi Tokuyama. Combinatorics and Algorithms on Low-Discrepancy Roundings of a Real Sequence. In *Automata, Languages and Programming – ICALP 2001*, volume 2076 of *Lecture Notes in Computer Science*, pages 166–177, 2001. doi: 10.1007/3-540-48224-5_14. URL https://link.springer.com/chapter/10.1007/3-540-48224-5_14.
- [21] Matthew Skala. The Well-Rounded Total. Author manuscript; explicitly not peer-reviewed, 2017. URL <https://ansuz.sooke.bc.ca/professional/rounding.pdf>.

- [22] Social Security Administration. POMS RS 00601.020: Rounding of Benefits. <https://secure.ssa.gov/poms.nsf/lnx/0300601020>, 2025. Accessed 3 October 2026. Rules depend on computation type and effective date. A small affine example is not a full SSA benefit calculation.
- [23] Alexey Solovyev, Charles Jacobsen, Zvonimir Rakamarić, and Ganesh Gopalakrishnan. Rigorous Estimation of Floating-Point Round-off Errors with Symbolic Taylor Expansions. In *FM 2015: Formal Methods*, volume 9109 of *Lecture Notes in Computer Science*, pages 532–550, 2015. doi: 10.1007/978-3-319-19249-9_33. URL https://soarlab.org/papers/2015_fm_sjrg.pdf.
- [24] Sven Verdoolaege. isl: An Integer Set Library for the Polyhedral Model. In *Mathematical Software – ICMS 2010*, volume 6327 of *Lecture Notes in Computer Science*, pages 299–302, 2010. doi: 10.1007/978-3-642-15582-6_49. URL <https://libisl.sourceforge.io/manual.pdf>.
- [25] H. P. Williams. The problem with integer programming. *IMA Journal of Management Mathematics*, 22(3):213–230, 2011. doi: 10.1093/imaman/dpq014. URL <https://personal.lse.ac.uk/williahp/pubs/WP%2010-118.pdf>. Journal issue 2011; author working paper dated 2010.
- [26] Kevin Woods. Presburger arithmetic, rational generating functions, and quasi-polynomials. *Journal of Symbolic Logic*, 80(2):433–449, 2015. doi: 10.1017/jsl.2015.4. URL https://www2.oberlin.edu/faculty/kwoods/research/Presburger_full.pdf.

A A sharp aggregation envelope

Lemma 5. *For arbitrary rational operands $y_1, \dots, y_n$ on a shared quantum q , the largest absolute nearest-even line/total discrepancy is $q\text{he}(n/2)$. For FLOOR or CEILING it is $(n - 1)q$.*

Proof. Normalize $q = 1$. For floor, write $y_i = k_i + f_i$ with $0 \leq f_i < 1$. The signed line-minus-total discrepancy is $-\lfloor \sum_i f_i \rfloor$, ranging from $-(n - 1)$ to zero. Rational fractions sufficiently near one attain the lower endpoint. Ceiling follows by negation.

For nearest-even set $k_i = \text{he}(y_i)$ and $f_i = y_i - k_i \in [-1/2, 1/2]$. If $|f_i| = 1/2$, k_i is even. Put $K = \sum_i k_i$, $F = \sum_i f_i$. The discrepancy is $K - \text{he}(K + F)$. If $n = 2h$, $|F| \leq h$ and nearest rounding cannot select an integer beyond $K \pm h$. If $n = 2h + 1$, the strict interior $|F| < h + 1/2$ gives absolute discrepancy at most h . At either boundary, all local residuals are the same half tie, so K is even; the outer tie gives h if h is even and $h + 1$ if h is odd. This is exactly $\text{he}(n/2)$. Choosing every operand $1/2$, or every operand $-1/2$, attains the endpoints. Rescale by q . $\square$

This elementary envelope is included for comparison, not as a novelty claim. Restricted input grids can give strictly smaller attainable extrema.